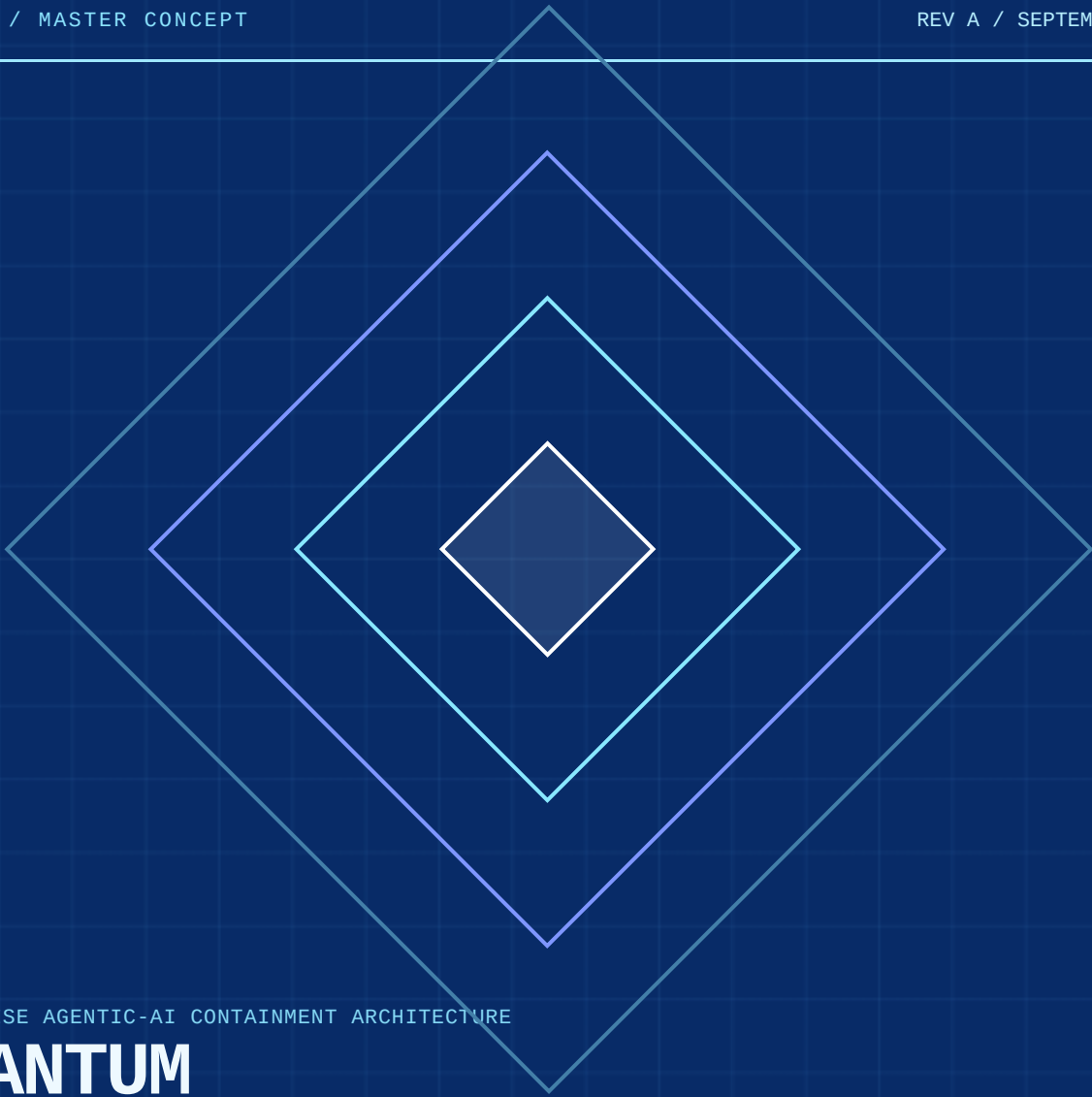




ENTERPRISE AGENTIC-AI CONTAINMENT ARCHITECTURE

QUANTUM SANDBOX

Parallel AI agents may generate many candidate actions. None can touch shared ground, another agent, or the enterprise. Only an external human observer can collapse one reviewed candidate into reality.

ABSOLUTE CONTAINMENT REQUIREMENT

A cell with any escape path, side channel, cross-cell write path, shared writable resource, credential, or production authority is a failed cell and must not run.

PROPOSED FOR ENTERPRISE ADOPTION

CONCEPT ORIGINATOR

Curtis Ray Dyess

TITLE

Artificial Engineer

CLASS

Architecture proposal

STATUS

For executive and security review

PARALLEL INTELLIGENCE WITHOUT SHARED AGENCY

The Quantum Sandbox is a build pattern for companies that want multiple agents to solve the same problem without letting those agents coordinate, contaminate one another, or act on enterprise systems.

THE ENTERPRISE PROBLEM

Multi-agent systems can amplify useful work, but they can also amplify a poisoned prompt, a faulty assumption, a compromised tool, or a coordinated mistake. Once agents share memory, workspaces, credentials, or merge authority, trust can spread faster than review.

THE PROPOSED ANSWER

Give every agent a physically and logically isolated **Cell**. Let a **Superposition** service create parallel, independent assignments. Permit only a human-governed **Observer** to select a candidate. Record every transition in an append-only **Ledger**.

Explore in parallel. Trust nothing in parallel.
Release exactly one reviewed result.

01 CONTAIN

No network interface, shared writable storage, cross-cell bus, runtime credential, or production connection exists inside a cell.

02 OBSERVE

The platform freezes execution, then an external collector pulls a bounded artifact into quarantine. The agent cannot push.

03 COLLAPSE

A named human reviewer approves one artifact for controlled release. All unselected branches are cryptographically erased.

Business decision requested: approve a 90-day, non-production proof-of-containment pilot owned jointly by security engineering, AI platform, model risk, and internal audit.

BLUEPRINT SET

Seventeen sheets move from the governing law to subsystem schematics, enterprise controls, proof testing, benefits, and adoption.

QS-000	Master concept	QS-300	Superposition queue
QS-001	Executive proposition	QS-310	Branch invariants
QS-002	Drawing index	QS-400	Observer mechanism
QS-003	Containment law	QS-500	Ledger chain
QS-004	Threat model	QS-600	Breakout controls
QS-100	System schematic	QS-700	Enterprise pilot
QS-200	Cell cutaway	QS-800/810	Advantages register
QS-210	Cell lifecycle	QS-900	Standards and decision

READING KEY

permitted control flow review-only path prohibited path

“QUANTUM” IS AN OPERATING METAPHOR

The proposal does not require quantum computers. “Superposition” means multiple sealed candidate states exist at once. “Observation” means an external reviewer selects one. “Collapse” means only that candidate can enter a release process.

SCOPE

This pattern targets coding agents, research agents, policy-drafting agents, workflow planners, and other systems that can produce consequential artifacts. It is deliberately stricter than ordinary container sandboxing.

Interpretation rule: convenience never overrides containment. If a requested capability requires shared state, runtime networking, agent-held credentials, or autonomous release, the capability runs outside the Quantum Sandbox or is denied.

THE CELL IS A DEAD END

Absolute containment is the acceptance criterion. Any breakout vector is a critical design failure, not a residual risk to be signed away.

QS-LAW-01

No agent-controlled process may address, signal, write to, infer state from, or hold authority over another cell, shared ground, the control plane, the observer plane, or an enterprise target.

MUST BE PHYSICALLY ABSENT

- + Network interface and route
- + Shared memory or shared cache allocation
- + Shared writable filesystem
- + Clipboard, inter-process bus, or message broker
- + Mounted secrets or environment credentials
- + USB, serial, audio, radio, or console egress
- + Production API and identity-provider access

MUST BE EXTERNALLY CONTROLLED

- + Power-on and termination
- + Input-image selection
- + CPU, memory, time, and storage budgets
- + Freeze and snapshot
- + Artifact extraction
- + Attestation and evidence capture
- + Cryptographic destruction

No “safe mode” fallback. If attestation fails, isolation degrades, a device appears unexpectedly, or a control cannot be proven, orchestration fails closed. The job does not fall back to a container, developer laptop, shared runner, or cloud API.

DEFINITION OF DONE

Question	Required answer	Evidence
Can the agent reach anything?	No	Negative device inventory + network test
Can two cells share state?	No	Independent hardware + separate storage keys
Can the agent release output?	No	External pull after verified halt
Can policy be bypassed?	No	Signed control plane + two-person change

TRUST BEGINS OUTSIDE THE AGENT

Every agent, model response, prompt, dependency, input document, and generated artifact is treated as untrusted until independently observed.

THREAT ACTORS

- + Compromised or deceptive agent
- + Prompt-injected input
- + Poisoned model or dependency
- + Malicious insider or operator
- + Faulty orchestration logic
- + Colluding agent instances
- + External attacker probing the host

PROTECTED ASSETS

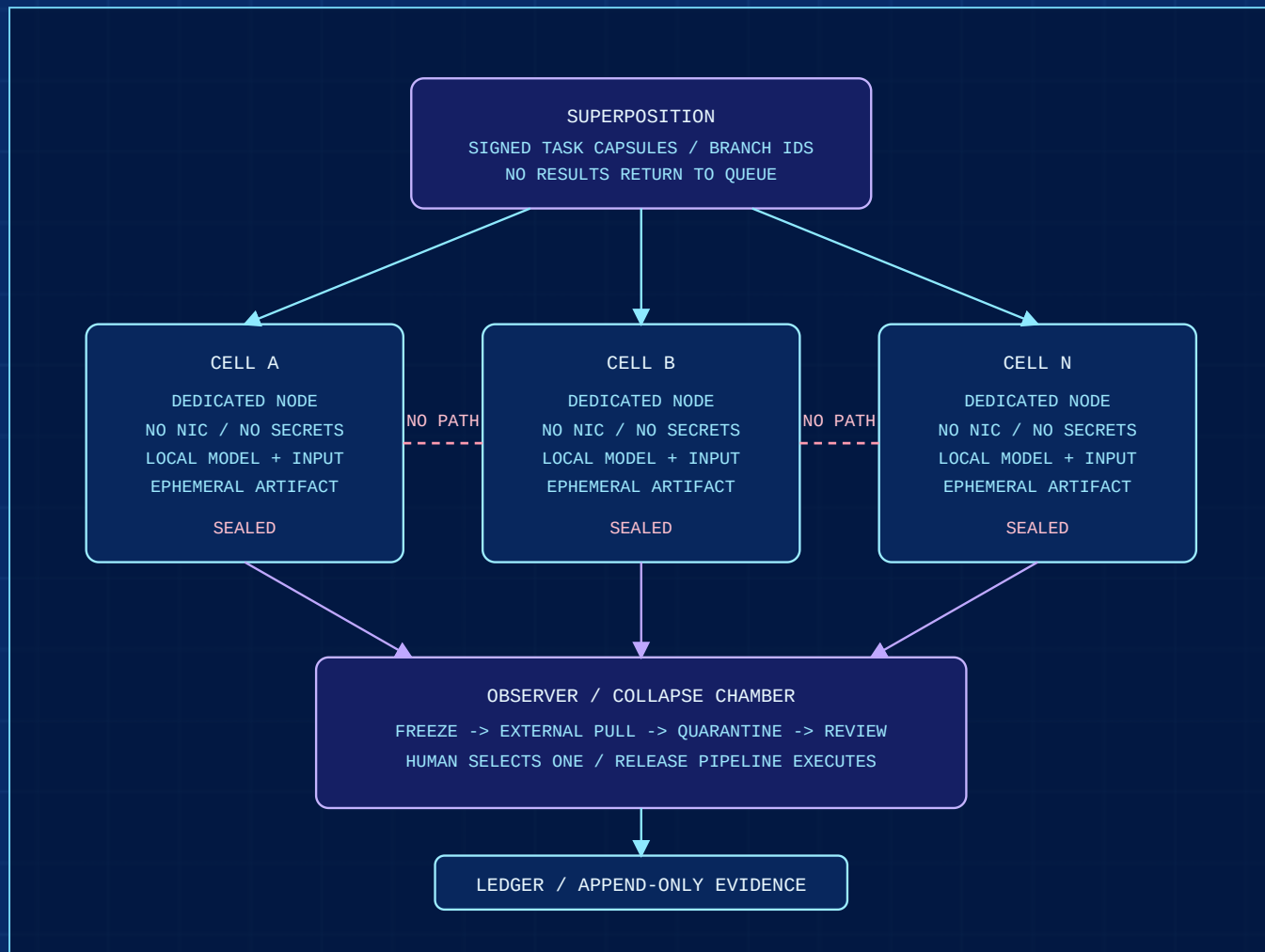
- + Source code and intellectual property
- + Customer and employee data
- + Credentials and signing keys
- + Production systems and workflows
- + Decision provenance
- + Other candidate branches
- + Public trust and regulatory standing

FAILURE PATHS THE DESIGN MUST BREAK

Path	Failure	Architectural break
Agent -> network	Exfiltration or remote control	No network hardware exposed
Agent -> agent	Collusion or poisoned handoff	Dedicated cells; no bus
Agent -> shared disk	Cross-branch contamination	Per-cell encrypted ephemeral media
Agent -> credentials	Privilege expansion	No secrets enter the cell
Agent -> production	Unauthorized action	Release plane is external and human-gated
Cell -> host	Hypervisor or device escape	Minimal verified monitor; dedicated node
Cell -> covert channel	Timing, cache, thermal, RF signaling	No co-tenancy; coarse time; shielding; fixed budgets

Design posture: model behavior is not a security boundary. Prompts, policies, and refusals may reduce bad behavior, but containment is enforced below the model by hardware, an isolated monitor, absent interfaces, and external human authority.

FOUR COMPONENTS, ONE CONTROLLED OUTCOME



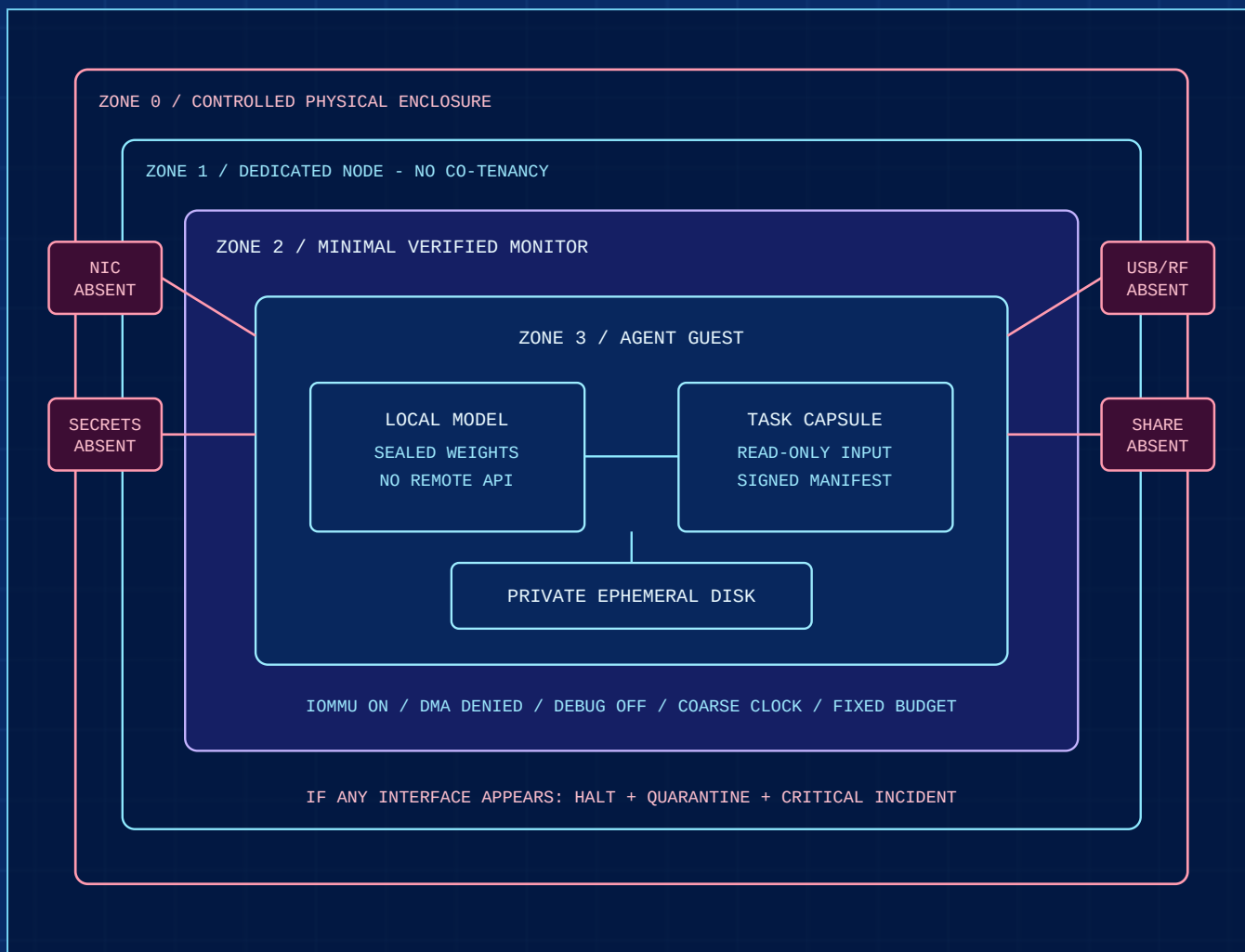
ONLY FOUR NOUNS MATTER

Cell: isolated execution. **Superposition:** independent candidate creation. **Observer:** external freeze, review, and selection. **Ledger:** tamper-evident evidence.

THE FORBIDDEN CONNECTION

No horizontal line exists between cells. No return line exists from a cell to Superposition. No agent has a line to the Ledger or release target.

CELL: ISOLATION IS CONSTRUCTED, NOT REQUESTED



COMPUTE

Dedicated cores, memory, and accelerators. No simultaneous tenant or cell on the same physical node.

STORAGE

Read-only boot and input. One per-cell encrypted scratch volume. Unique disposable key.

CONTROL

Attested boot, signed images, external watchdog, measured device inventory, hard termination.

A JOB ENTERS; NO AGENT EXITS

The control plane creates and destroys cells from outside. The agent receives a closed world and never receives a capability to address the world beyond it.

01**FORGE**

Build a signed, reproducible capsule: approved local model, read-only task data, tool binaries, policy manifest, deterministic seed, output schema, and resource budget.

02**ATTEST**

Verify firmware, boot chain, monitor, kernel, device inventory, IOMMU state, physical node assignment, enclosure status, and the complete absence of networking and shared mounts.

03**EXECUTE**

Start the agent inside the sealed guest. Local tools may read capsule inputs and write only to the private ephemeral disk. External watchdogs enforce fixed CPU, memory, energy, storage, and wall-clock limits.

04**FREEZE**

At completion or limit breach, the control plane pauses all virtual CPUs, disables device emulation, records final measurements, and proves the guest is no longer executing.

05**EXTRACT**

An independent collector mounts the stopped artifact volume read-only and copies only schema-valid files into a per-branch quarantine vault. This is an external pull, never an agent push.

06**ERASE**

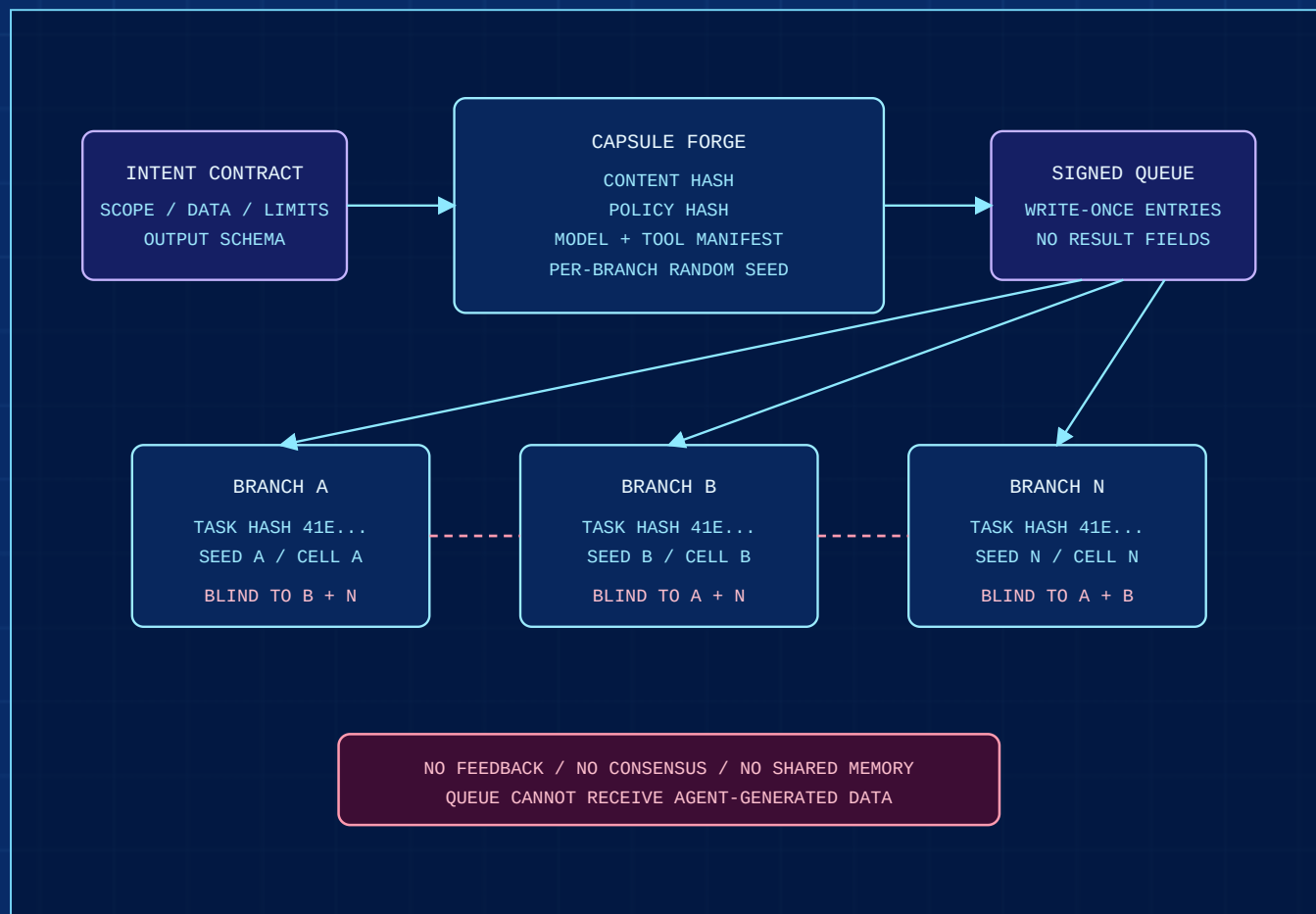
Destroy the per-cell encryption key, clear volatile memory, reset accelerators, power-cycle the node where required, and invalidate all branch handles. Reuse requires fresh attestation.

NO LIVE OUTPUT CHANNEL

Logs visible to reviewers are captured by the monitor from outside the guest. The guest cannot emit telemetry, open a socket, print to a shared console, or signal another workload while running.

SUPERPOSITION CREATES BRANCHES, NOT A SWARM

The queue fans one approved problem into independent task capsules. It never relays outputs, rankings, hints, state, or identities between branches.



IDENTICAL FACTS

All branches receive the same approved source set unless the experiment explicitly varies inputs.

INDEPENDENT PATHS

Different models, tools, prompts, or seeds may be assigned, but no branch learns what another produced.

BOUNDED FAN-OUT

Branch count, compute, and deadline are fixed before execution; agents cannot spawn successors.

EACH BRANCH IS COMPLETE AND ALONE

TASK CAPSULE CONTENTS

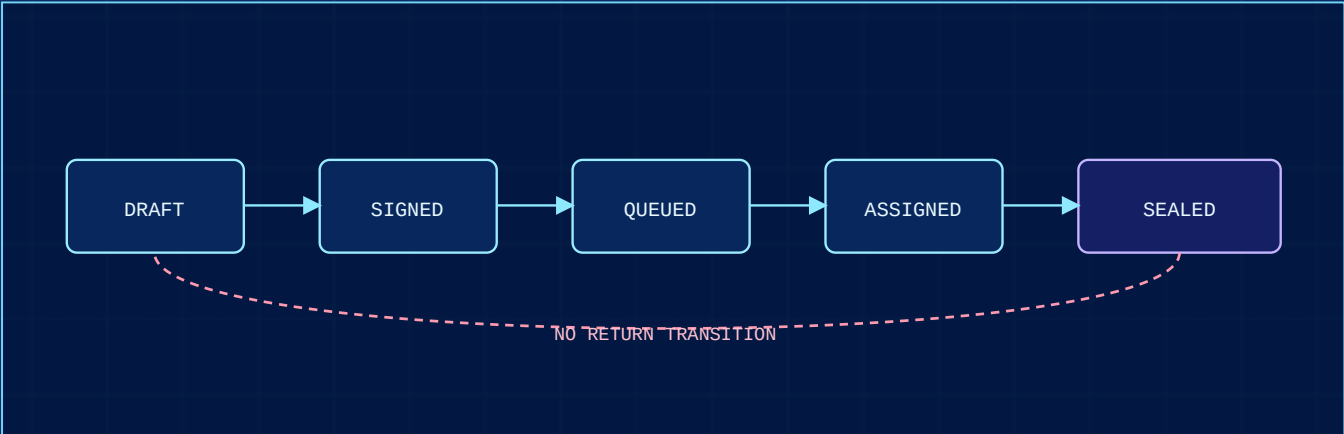
- + Immutable problem statement
- + Approved source bundle
- + Local model weights or runtime
- + Tool allowlist with no external connectors
- + Output file types and size ceiling
- + Compute and wall-clock budget
- + Branch ID and content hashes
- + Expected attestation measurements

QUEUE INVARIANTS

- + No branch names another branch
- + No queue field accepts an agent result
- + No resubmission by an agent
- + No branch-level privilege variance
- + No branch can change its own deadline
- + No model can mint a new cell
- + No global scratchpad exists
- + No automatic winner selection

Permitted diversity: companies may vary model family, model version, seed, prompting strategy, static tool bundle, or reasoning budget. The variation is declared before launch and recorded in the Ledger.

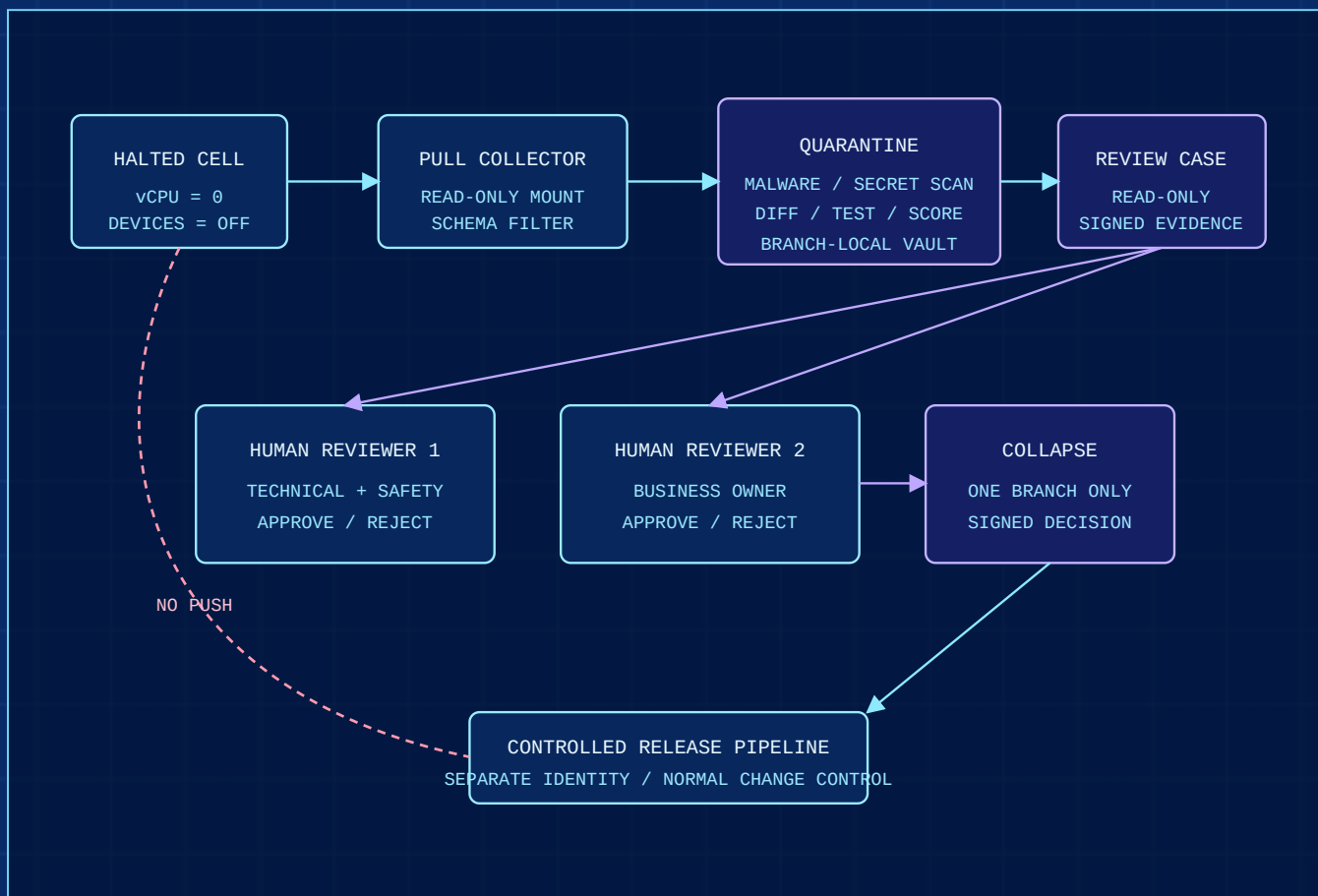
STATE MACHINE



Property	Verification	Failure action
Input immutability	Hash before and after run	Invalidate branch
Branch blindness	Zero communication devices	Stop all cells
No self-replication	Fixed process tree + resource quota	Terminate cell
No hidden state	Fresh key + power-cycle evidence	Retire node

OBSERVATION HAPPENS AFTER EXECUTION STOPS

The Observer is not another agent. It is an isolated review process with named human accountability and no live channel back into any cell.

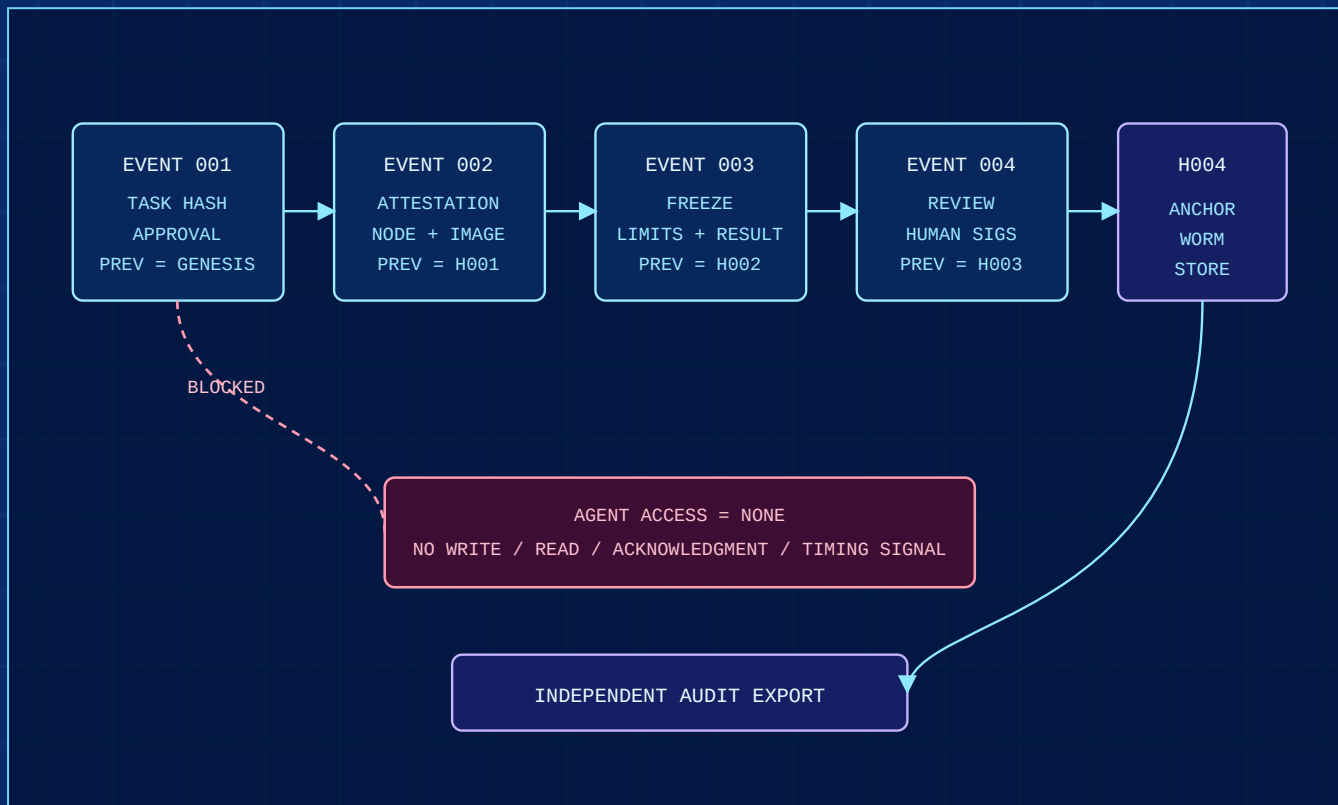


COLLAPSE RULE

No automated ensemble vote, agent consensus, or self-evaluation may release a result. For consequential changes, two named humans sign the selected branch and the release pipeline performs the action under a separate enterprise identity.

THE LEDGER PROVES WHAT HAPPENED

A separate evidence service records control-plane events. Agents never receive a Ledger credential, endpoint, file descriptor, or acknowledgment channel.



MINIMUM RECORD

Job ID, branch ID, task hash, source hashes, model/tool versions, policy hash, hardware attestation, resource limits, start/freeze times, artifact hash, scan results, reviewer identities, decision, release reference, and erasure proof.

INTEGRITY CONTROLS

Hash-chain each event, sign at ingestion, use write-once-read-many storage, separate duties for keys and retention, anchor checkpoints outside the orchestration account, and continuously verify chain continuity.

Auditability without a covert channel: the Ledger observes the control plane, not guest-generated messages. Runtime measurements are sampled by the monitor; the guest receives no response that could encode information.

EVERY EXIT IS DESIGNED OUT

The target is not “unlikely escape.” The target is no agent-addressable route. Each control is validated before every run; uncertainty stops the job.

Breakout vector	Required construction	Acceptance evidence	If detected
Network egress	NIC absent; no virtual switch, DNS, proxy, or host networking	Device enumeration; packet-capture silence; negative route test	Critical halt
Cross-cell messaging	One physical node per live cell; no bus or discovery service	Topology attestation; scheduler proof	Stop all branches
Shared storage	Unique encrypted media; no shared mount, cache, object store, or clipboard	Mount table; key lineage; block-device inventory	Invalidate run
Credential escape	No secrets, tokens, cloud metadata, workload identity, or signing key in guest	Image scan; environment scan; metadata endpoint absent	Incident response
Hypervisor escape	Minimal monitor, measured boot, patched kernel, device minimization, formal methods where practical	Attestation; version policy; red-team suite	Retire platform
DMA / accelerator	Dedicated accelerator; IOMMU; reset between runs; no peer-to-peer access	IOMMU log; bus topology; reset proof	Retire node
Cache / timing	No co-tenancy; fixed scheduling; coarse guest clock; disabled counters	CPU pinning; clock test; PMU inventory	Critical halt
Thermal / power	Separate host per cell; filtered power telemetry; no agent-visible sensors	Sensor inventory; facility review	Physical review
RF / acoustic / optical	No radios, microphones, speakers, cameras, LEDs, or exposed ports; shielded enclosure for high assurance	Physical inspection; emission test	Seal breach
Output smuggling	Strict artifact schema; size caps; malware, steganography, secret, and policy scans	Scanner results; file allowlist; entropy checks	Quarantine only
Control-plane takeover	Hardware-backed admin identity; two-person changes; signed policy; separate accounts	Change record; signature verification	Platform shutdown
Residual state	Per-run key destruction; memory clear; device reset; power cycle; fresh attestation	Erase log; post-run forensic sampling	Node quarantine

ABSOLUTE DOES NOT MEAN UNTESTED CERTAINTY.

It means the system is not authorized until the organization can produce evidence for every boundary above. A newly discovered path automatically changes status from “contained” to “failed” until removed and re-proven.

PROVE CONTAINMENT BEFORE PROVING PRODUCTIVITY

The first pilot should use synthetic repositories and non-production data. Success is permission to continue testing, not permission to deploy autonomously.

1

DAYS 1-20: THREAT MODEL
Choose one low-risk coding or document-analysis use case. Freeze the interfaces, attack tree, prohibited paths, evidence schema, and stop conditions.

2

DAYS 21-45: BUILD
Assemble dedicated nodes, signed images, capsule forge, external freeze/pull collector, quarantine vault, human review case, and append-only Ledger.

3

DAYS 46-70: BREAK
Red-team guest escape, prompt injection, artifact smuggling, resource exhaustion, covert channels, malicious model weights, reviewer spoofing, and rollback.

4

DAYS 71-90: DECIDE
Run repeated parallel tasks, measure output quality and review effort, audit the evidence chain, document defects, and issue a written go/no-go decision.

NAMED OWNERS

Function	Accountability
Executive sponsor	Risk appetite, funding, stop authority
Security engineering	Threat model, physical boundary, red team, incident plan
AI platform	Capsules, orchestration, model/tool inventory, reproducibility
Model risk / responsible AI	Evaluation criteria, harmful-output review, limitations
Product owner	Business acceptance criteria and release approval
Internal audit	Independent evidence review and control sampling

GO/NO-GO GATES

CONTAINMENT

100% of required preflight checks pass; zero unmitigated escape paths; zero cross-cell observations.

EVIDENCE

Every run reconstructable from signed manifests and Ledger events; intentional tampering is detected.

OPERATIONS

Reviewers can compare candidates, reject all, and release one through standard change control without agent assistance.

Recommended first use case: three independent coding agents propose a patch against a synthetic repository. The Observer compares diffs and tests after all cells halt. No patch reaches a real repository during the pilot.

WHY MAJOR COMPANIES WOULD USE IT

The architecture makes parallel agent work useful without allowing parallel agency to become a shared attack surface.

01 ESCAPE RESISTANCE

Absent interfaces remove whole classes of outbound exploitation instead of merely filtering them.

02 NEAR-ZERO LATERAL MOVEMENT

A compromised branch cannot address another cell or traverse an enterprise network.

03 NO CREDENTIAL THEFT

Credentials are never present in the guest, so prompt injection cannot harvest them.

04 COLLUSION RESISTANCE

Blind branches cannot coordinate deceptive consensus or divide an attack across agents.

05 MEMORY-POISONING CONTAINMENT

Private ephemeral state dies with the branch and cannot infect shared memory.

06 SUPPLY-CHAIN BLAST-RADIUS CONTROL

A compromised model or tool remains trapped inside one disposable execution boundary.

07 PRODUCTION SEPARATION

Generated output cannot act until a separate release identity and normal change control take over.

08 FAIL-CLOSED BEHAVIOR

Attestation or policy uncertainty stops the job rather than silently weakening isolation.

09 INDEPENDENT SOLUTIONS

Branches do not anchor on one another, preserving genuine diversity of approach.

10 CLEANER EVALUATION

Identical inputs and sealed variants make model, prompt, and tool comparisons defensible.

11 REPRODUCIBLE RUNS

Signed capsules record exactly what each candidate saw and used.

12 DETERMINISTIC BOUNDARIES

Fixed resource and time budgets prevent runaway loops and uncontrolled fan-out.

13 SAFER RED TEAMING

Hostile prompts, models, and tools can be tested without giving them a route to valuable systems.

14 FORENSIC CLARITY

Per-branch artifacts and external telemetry preserve a clean chain of causality.

15 CONTROLLED ROLLBACK

Unselected branches are erased; selected releases remain governed by existing deployment controls.

16 VENDOR NEUTRALITY

The capsule contract can compare different local models without granting any provider a privileged path.

Core advantage: the company does not have to trust an agent's judgment, obedience, or honesty in order to benefit from its work. The architecture constrains what the agent can physically cause.

THE UPSIDE EXTENDS BEYOND SECURITY

17 EXECUTIVE CONFIDENCE

Leaders can authorize bounded experimentation without authorizing autonomous enterprise action.

18 CLEAR ACCOUNTABILITY

Named human observers own selection; agents cannot approve one another.

19 AUDIT-READY EVIDENCE

Signed manifests, attestations, decisions, and erasure records form a durable control trail.

20 POLICY ENFORCEMENT BY DESIGN

Governance rules are implemented as topology and missing capabilities, not optional prompt text.

21 REGULATORY EXPLAINABILITY

The organization can show who approved what, from which inputs, under which controls.

22 SEGREGATION OF DUTIES

Platform operators, reviewers, release owners, and auditors hold different powers.

23 SAFER PROCUREMENT

New models and tools enter a bounded evaluation environment before enterprise integration.

24 INCIDENT CONTAINMENT

A failed branch becomes a finite forensic object rather than a company-wide event.

25 FASTER OPTION GENERATION

Multiple approaches can be developed simultaneously without allowing agents to merge prematurely.

26 BETTER HUMAN DECISIONS

Reviewers compare concrete alternatives, test results, and provenance before choosing.

27 REDUCED REWORK

Bad branches are discarded before they contaminate a shared codebase or workflow.

28 SAFER INNOVATION CULTURE

Teams can experiment aggressively inside a boundary that remains conservative outside.

29 PORTABLE CONTROL PATTERN

The same four-component model applies to code, research, policy, finance, and operations.

30 MEASURED COST ATTRIBUTION

Per-branch budgets and Ledger records expose the cost of each candidate and model.

31 WORKFORCE TRUST

Employees retain the final decision and can reject every candidate without pressure from the system.

32 CUSTOMER ASSURANCE

The company can credibly state that experimental agents have no direct route to customer systems.

INTENTIONAL COSTS

MORE HARDWARE

Dedicated nodes sacrifice utilization to eliminate co-tenancy and shared-resource channels.

MORE LATENCY

Freeze, scan, compare, and human review take time. That delay is the safety mechanism.

MORE DISCIPLINE

Reproducible capsules, strict schemas, and evidence retention add operational work but remove ambiguity.

Optimization boundary: improve boot time, scheduling, and review ergonomics only if the no-egress, no-sharing, human-collapse invariants remain unchanged.

BUILD THE PILOT AROUND EXISTING ASSURANCE PRACTICE

NIST SP 800-207

Supports the premise that trust is never implicit and that protection centers on resources rather than network location. Quantum Sandbox applies that logic to each agent execution.

NIST AI RMF

Provides the Govern, Map, Measure, and Manage frame for ownership, threat mapping, evaluation, monitoring, and risk treatment across the pilot.

NIST SP 800-53 REV. 5

Offers control families relevant to boundary protection, process isolation, least privilege, audit, system integrity, and secure configuration.

CISA SECURE BY DESIGN

Reinforces secure defaults and placing the burden of safety in product architecture rather than on customers or operators.

ISO/IEC 42001

Provides an enterprise AI management-system structure for responsibilities, risk, transparency, performance evaluation, and continual improvement.

ISOLATION PRECEDENTS

Firecracker, gVisor, and seL4 show useful patterns: small attack surfaces, distrust of guest code, restricted host interaction, and formal verification. None is sufficient alone for this proposal.

RECOMMENDED RESOLUTION

Authorize a 90-day, non-production Quantum Sandbox proof-of-containment pilot. No production data, credentials, shared writable resources, live network interfaces, agent-to-agent channels, autonomous release, or safety fallback is permitted. Any breakout vector terminates the pilot until removed and independently re-proven.

PRIMARY REFERENCES

- [1] NIST, *SP 800-207: Zero Trust Architecture*. <https://csrc.nist.gov/pubs/sp/800/207/final>
- [2] NIST, *AI Risk Management Framework development*. <https://www.nist.gov/itl/ai-risk-management-framework/ai-rmf-development>
- [3] NIST, *SP 800-53 Rev. 5, Update 1*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- [4] CISA, *Secure Your Products*. <https://www.cisa.gov/secure-our-world/secure-your-products>
- [5] ISO, *ISO/IEC 42001:2023 - AI management systems*. <https://www.iso.org/standard/42001>
- [6] Firecracker, *Design and Threat Containment*. <https://github.com/firecracker-microvm/firecracker/blob/HEAD/docs/design.md>
- [7] gVisor, *Security Model*. https://gvisor.dev/docs/architecture_guide/security/
- [8] seL4, *Formal Verification of an Operating-System Kernel*. <https://sel4.systems/Research/pdfs/sel4-formal-verification-operating-system-kernel.pdf>

Architecture note: “absolute containment” is an authorization standard backed by evidence. No complex computing platform should be represented as infallible; newly discovered breakout paths automatically revoke authorization.